Función Generadora de Dimensión Fecha

Esta función personalizada de Power Query genera automáticamente una tabla calendario (dimensión fecha) completa con múltiples columnas de análisis temporal. Solo requiere dos parámetros: fecha de inicio y fecha final.

```
(FechaInicio as date, FechaFinal as date) as table =>
let
    // Validar que la fecha inicial sea menor o igual a la final
    ValidarFechas = if FechaInicio <= FechaFinal then true else error "La fecha de inicio debe ser menor o igual a la fecha final",

    // Generar lista de fechas
    CantidadDias = Duration.Days(FechaFinal - FechaInicio) + 1,
    ListaFechas = List.Dates(FechaInicio, CantidadDias, #duration(1,0,0,0)),

    // Convertir a tabla
    TablaFechas = Table.FromList(ListaFechas, Splitter.SplitByNothing(), {"Fecha"}, null, ExtraValues.Error),

    // Formatear la columna Fecha como date
    FormatearFecha = Table.TransformColumnTypes(TablaFechas, {{"Fecha", type date}}),

    // Agregar columnas adicionales
    AgregarAño = Table.AddColumn(FormatearFecha, "Año", each Date.Year([Fecha]), Int64.Type),
    AgregarMes = Table.AddColumn(AgregarAño, "Mes", each Date.Month([Fecha]), Int64.Type),
    AgregarNombreMes = Table.AddColumn(AgregarMes, "NombreMes", each Text.Start(Date.MonthName([Fecha]), 3), type text),
    AgregarDia = Table.AddColumn(AgregarNombreMes, "Dia", each Date.Day([Fecha]), Int64.Type),
    AgregarDiaSemana = Table.AddColumn(AgregarDia, "DiaSemana", each Date.DayOfWeek([Fecha], Day.Monday) + 1, Int64.Type),
    AgregarNombreDiaSemana = Table.AddColumn(AgregarDiaSemana, "NombreDiaSemana", each Text.Start(Date.DayOfWeekName([Fecha]), 3), type text),
    AgregarTrimestre = Table.AddColumn(AgregarNombreDiaSemana, "Trimestre", each Date.QuarterOfYear([Fecha]), Int64.Type),
    AgregarSemanaAño = Table.AddColumn(AgregarTrimestre, "SemanaAño", each Date.WeekOfYear([Fecha]), Int64.Type),

    // Agregar columnas de formato
    AgregarAñoMes = Table.AddColumn(AgregarSemanaAño, "AñoMes", each Text.From([Año]) & "-" & Text.PadStart(Text.From([Mes]), 2, "0"), type text),

    // Resultado final
    Resultado = AgregarAñoMes
in
    Resultado
```

```
(StartDate as date, EndDate as date) as table =>
let
    // Validate that start date is less than or equal to end date
    ValidateDates = if StartDate <= EndDate then true else error "Start date must be less than or equal to end date",

    // Generate list of dates
    DayCount = Duration.Days(EndDate - StartDate) + 1,
    DateList = List.Dates(StartDate, DayCount, #duration(1,0,0,0)),

    // Convert to table
    DateTable = Table.FromList(DateList, Splitter.SplitByNothing(), {"Date"}, null, ExtraValues.Error),

    // Transform the Date column as date
    TransformDate = Table.TransformColumnTypes(DateTable, {{"Date", type date}}),

    // Add additional columns
    AddYear = Table.AddColumn(TransformDate, "Year", each Date.Year([Date]), Int64.Type),
    AddMonth = Table.AddColumn(AddYear, "Month", each Date.Month([Date]), Int64.Type),
    AddMonthName = Table.AddColumn(AddMonth, "MonthName", each Text.Start(Date.MonthName([Date]), 3), type text),
    AddDay = Table.AddColumn(AddMonthName, "Day", each Date.Day([Date]), Int64.Type),
    AddDayOfWeek = Table.AddColumn(AddDay, "DayOfWeek", each Date.DayOfWeek([Date], Day.Monday) + 1, Int64.Type),
    AddDayName = Table.AddColumn(AddDayOfWeek, "DayName", each Text.Start(Date.DayOfWeekName([Date]), 3), type text),
    AddQuarter = Table.AddColumn(AddDayName, "Quarter", each Date.QuarterOfYear([Date]), Int64.Type),
    AddWeekOfYear = Table.AddColumn(AddQuarter, "WeekOfYear", each Date.WeekOfYear([Date]), Int64.Type),

    // Add format columns
    AddYearMonth = Table.AddColumn(AddWeekOfYear, "YearMonth", each Text.From([Year]) & "-" & Text.PadStart(Text.From([Month]), 2, "0"), type text),

    // Final result
    Result = AddYearMonth
in
    Result
```

Código M (Ingles)

```
(StartDate as date, EndDate as date) as table =>
let
    // Validate that start date is less than or equal to end date
    ValidateDates = if StartDate <= EndDate then true else error "Start date must be less than or equal to end date",

    // Generate list of dates
    DayCount = Duration.Days(EndDate - StartDate) + 1,
    DateList = List.Dates(StartDate, DayCount, #duration(1,0,0,0)),

    // Convert to table
    DateTable = Table.FromList(DateList, Splitter.SplitByNothing(), {"Date"}, null, ExtraValues.Error),

    // Transform the Date column as date
    TransformDate = Table.TransformColumnTypes(DateTable, {{"Date", type date}}),

    // Add additional columns
    AddYear = Table.AddColumn(TransformDate, "Year", each Date.Year([Date]), Int64.Type),
    AddMonth = Table.AddColumn(AddYear, "Month", each Date.Month([Date]), Int64.Type),
    AddMonthName = Table.AddColumn(AddMonth, "MonthName", each Text.Start(Date.MonthName([Date]), 3), type text),
    AddDay = Table.AddColumn(AddMonthName, "Day", each Date.Day([Date]), Int64.Type),
    AddDayOfWeek = Table.AddColumn(AddDay, "DayOfWeek", each Date.DayOfWeek([Date], Day.Monday) + 1, Int64.Type),
    AddDayName = Table.AddColumn(AddDayOfWeek, "DayName", each Text.Start(Date.DayOfWeekName([Date]), 3), type text),
    AddQuarter = Table.AddColumn(AddDayName, "Quarter", each Date.QuarterOfYear([Date]), Int64.Type),
    AddWeekOfYear = Table.AddColumn(AddQuarter, "WeekOfYear", each Date.WeekOfYear([Date]), Int64.Type),

    // Add format columns
    AddYearMonth = Table.AddColumn(AddWeekOfYear, "YearMonth", each Text.From([Year]) & "-" & Text.PadStart(Text.From([Month]), 2, "0"), type text),

    // Final result
    Result = AddYearMonth
in
    Result
```

Código M (Español)

```
(FechaInicio as date, FechaFinal as date) as table =>
let
   // Validar que la fecha inicial sea menor o igual a la final
   ValidarFechas = if FechaInicio <= FechaFinal then true else error "La fecha de inicio debe ser menor o igual a la fecha final",

   // Generar lista de fechas
   CantidadDias = Duration.Days(FechaFinal - FechaInicio) + 1,
   ListaFechas = List.Dates(FechaInicio, CantidadDias, #duration(1,0,0,0)),

   // Convertir a tabla
   TablaFechas = Table.FromList(ListaFechas, Splitter.SplitByNothing(), {"Fecha"}, null, ExtraValues.Error),

   // Formatear la columna Fecha como date
   FormatearFecha = Table.TransformColumnTypes(TablaFechas, {{"Fecha", type date}}),

   // Agregar columnas adicionales
   AgregarAño = Table.AddColumn(FormatearFecha, "Año", each Date.Year([Fecha]), Int64.Type),
   AgregarMes = Table.AddColumn(AgregarAño, "Mes", each Date.Month([Fecha]), Int64.Type),
   AgregarNombreMes = Table.AddColumn(AgregarMes, "NombreMes", each Text.Start(Date.MonthName([Fecha]), 3), type text),
   AgregarDia = Table.AddColumn(AgregarNombreMes, "Dia", each Date.Day([Fecha]), Int64.Type),
   AgregarDiaSemana = Table.AddColumn(AgregarDia, "DiaSemana", each Date.DayOfWeek([Fecha], Day.Monday) + 1, Int64.Type),
   AgregarNombreDiaSemana = Table.AddColumn(AgregarDiaSemana, "NombreDiaSemana", each Text.Start(Date.DayOfWeekName([Fecha]), 3), type text),
   AgregarTrimestre = Table.AddColumn(AgregarNombreDiaSemana, "Trimestre", each Date.QuarterOfYear([Fecha]), Int64.Type),
   AgregarSemanaAño = Table.AddColumn(AgregarTrimestre, "SemanaAño", each Date.WeekOfYear([Fecha]), Int64.Type),

   // Agregar columnas de formato
   AgregarAñoMes = Table.AddColumn(AgregarSemanaAño, "AñoMes", each Text.From([Año]) & "-" & Text.PadStart(Text.From([Mes]), 2, "0"), type text),

   // Resultado final
   Resultado = AgregarAñoMes
in
   Resultado
```